

Data Structures Assignment

Course: Data Structures

Assignment: AVL Trees and Binary Search Trees

Students: Shahar Cohen, Tal Shaubi

ID's: 213670573, 209410356

Date: 10/07/2026

1. Question 1

Since we are inserting in a sorted order from smallest to largest the keys to a regular BST, we will get a skewed tree with the height of $n - 1$, when every node doesn't have a left son, the right son is the successor and the parent is the predecessor.

1.1. Analysis

The time complexity for this experiment is $O(n^2)$ since for any iteration i we need to go down the entire tree thus will take $O(i)$ complexity. And by adding it all together we will get:

$$T(n) = \sum_{i=1}^n c \cdot i = c \cdot \sum_{i=1}^n i = c \cdot \frac{(n+1)n}{2} = O(n^2)$$

2. Question 2

For an AVL Tree with n nodes we will get a tree with the height of $O(\log(n))$ since this is a AVL Tree that maintains a $|\text{BF}| < 2$ which allows the tree to have a logarithmic height compared to the number of nodes. As for the insertion, for the i 'th insertion, we will need to travel down the tree which will take us $O(\log(i))$ because of the height of the tree so far, then go up back the tree in order to rebalance and to update the height of the nodes. This will also take $O(\log(i))$ because we will do at most 2 rotations per rebalance and a height change which is with constant time complexity.

2.1. Analysis

The time complexity for this experiment is $O(n \log(n))$. Each insertion goes down the tree which takes $\log(i)$ and then goes back up the tree which also take $\log(i)$. Therefore the analysis will look like this:

$$T(n) = \sum_{i=1}^n \log(i) = \log(\prod_{i=1}^n i) = O(n \log(n))$$

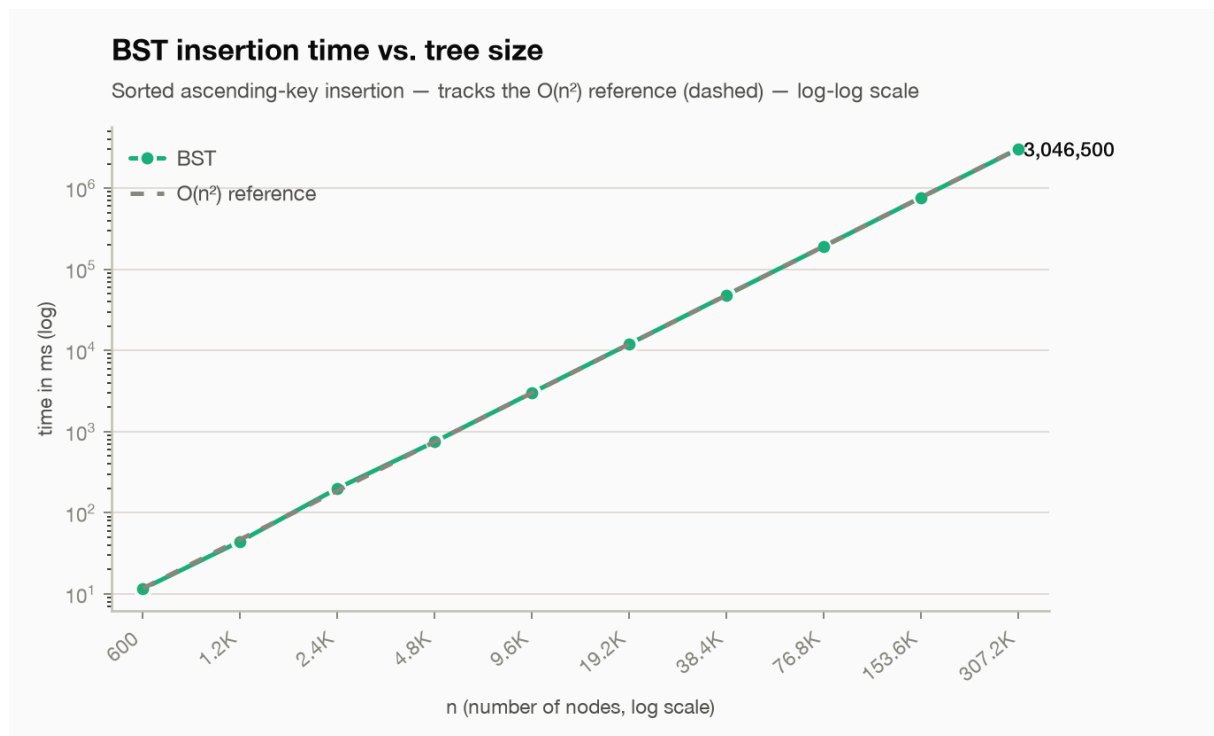
3. Question 3

3.1. Sorted Insertion - Regular BST

3.1.1. Table

Iteration	Size	Height	Rotations	Height Changes	Run time (actions)	Run time (ms)
1	600	599	-	-	180300	11.564
2	1200	1199	-	-	720600	44.543
3	2400	2399	-	-	2881200	200.373
4	4800	4799	-	-	11522400	749.517
5	9600	9599	-	-	46084800	3017.151
6	19200	19199	-	-	184329600	12016.072
7	38400	38399	-	-	737299200	48151.566
8	76800	76799	-	-	2949158400	191680.071
9	153600	153599	-	-	11796556800	766912.287
10	307200	307199	-	-	47186073600	3046499.635

3.1.2. Runtime as a function of n graph

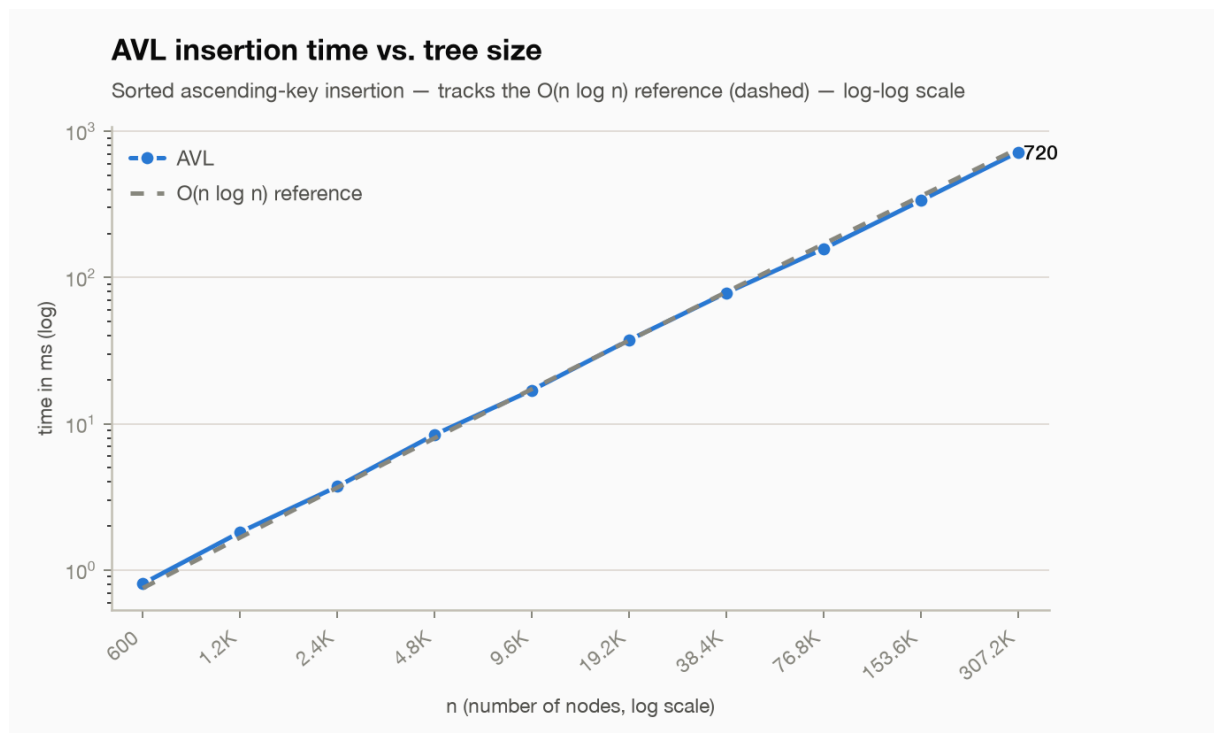


3.2. Sorted Insertion - AVL Tree

3.2.1. Table

Iteration	Size	Height	Rotations	Height Changes	Run time (actions)	Run time (ms)
1	600	9	590	1186	7353	0.808
2	1200	10	1189	2385	15927	1.816
3	2400	11	2388	4784	34277	3.727
4	4800	12	4787	9583	73379	8.439
5	9600	13	9586	19182	156385	16.981
6	19200	14	19185	38381	332999	37.363
7	38400	15	38384	76780	702429	78.610
8	76800	16	76783	153579	1481691	157.310
9	153600	17	153582	307178	3117017	336.080
10	307200	18	307181	614377	6541271	720.066

3.2.2. Runtime as a function of n graph

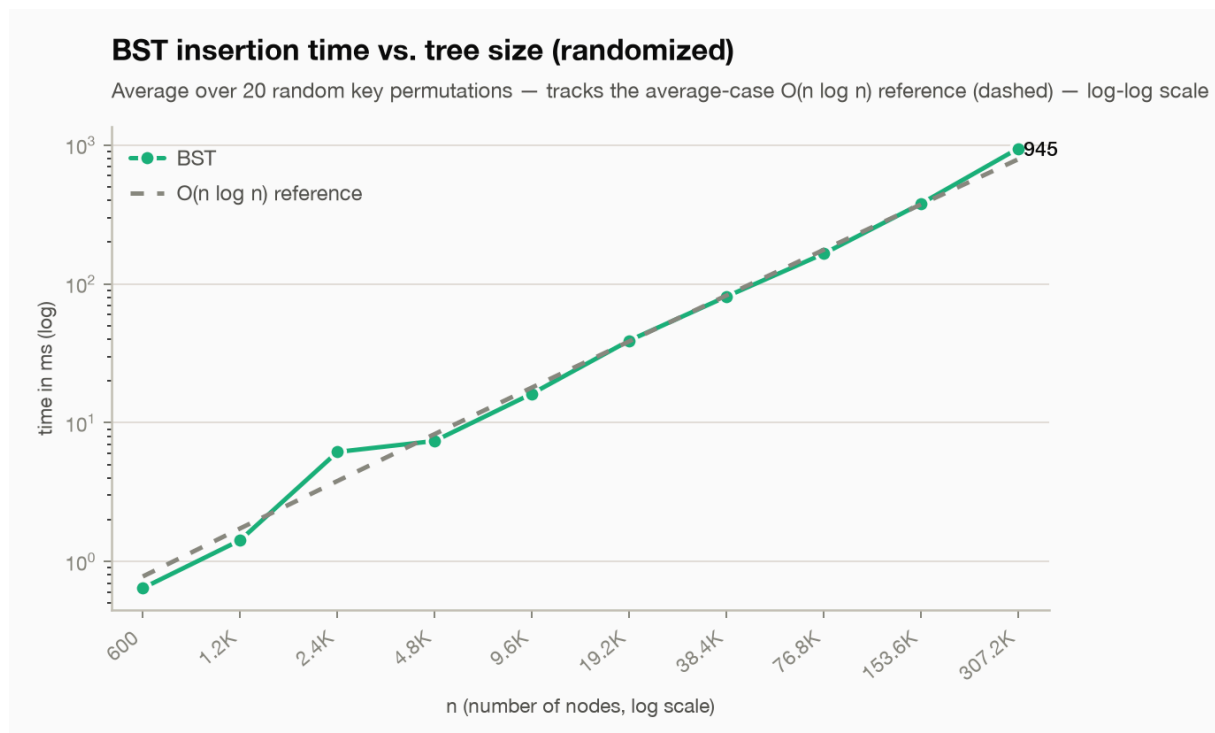


3.3. Random Insertion - Regular BST

3.3.1. Table

Iteration	Size	Height	Rotations	Height Changes	Run time (actions)	Run time (ms)
1	600	19.55	-	-	6598.9	0.644
2	1200	21.8	-	-	14859.7	1.424
3	2400	23.75	-	-	32740.9	6.154
4	4800	27.45	-	-	72973.4	7.389
5	9600	29.85	-	-	157577.3	16.183
6	19200	32.85	-	-	343595.5	39.045
7	38400	35.85	-	-	747875.95	81.002
8	76800	38.05	-	-	1570674.15	164.962
9	153600	41.2	-	-	3404252.15	378.157
10	307200	44.65	-	-	7196368.95	944.617

3.3.2. Runtime as a function of n graph

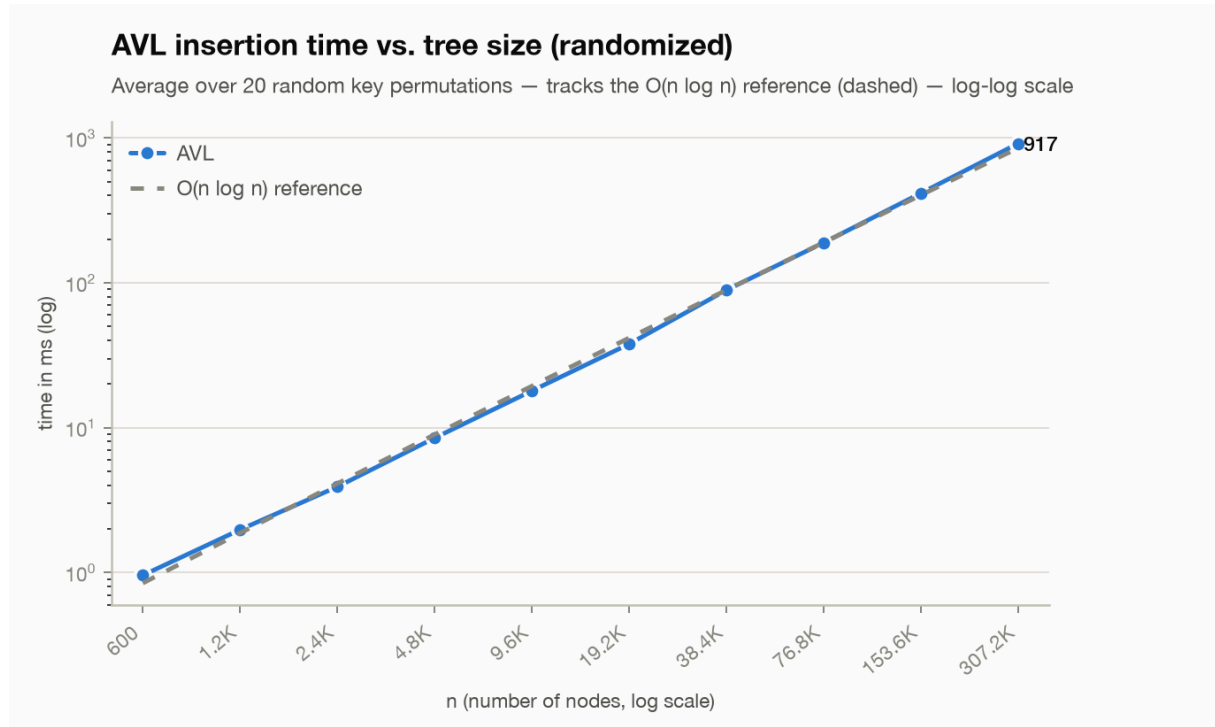


3.4. Random Insertion - AVL Tree

3.4.1. Table

Iteration	Size	Height	Rotations	Height Changes	Run time (actions)	Run time (ms)
1	600	10.05	411.15	1061.6	6861.1	0.957
2	1200	11.15	840.55	2131.25	14975.95	1.974
3	2400	12.4	1665.75	4264.95	32342.75	3.907
4	4800	13.75	3341.85	8523.25	69620.3	8.485
5	9600	15.0	6658.4	17096.15	148902.85	17.959
6	19200	16.0	13391.7	34260.85	317510.0	37.815
7	38400	17.05	26848.45	68574.5	674731.5	89.135
8	76800	18.55	53615.0	137039.1	1426042.0	189.326
9	153600	19.85	107184.4	274203.8	3009953.3	416.285
10	307200	21.0	214539.65	548395.4	6331076.1	917.174

3.4.2. Runtime as a function of n graph



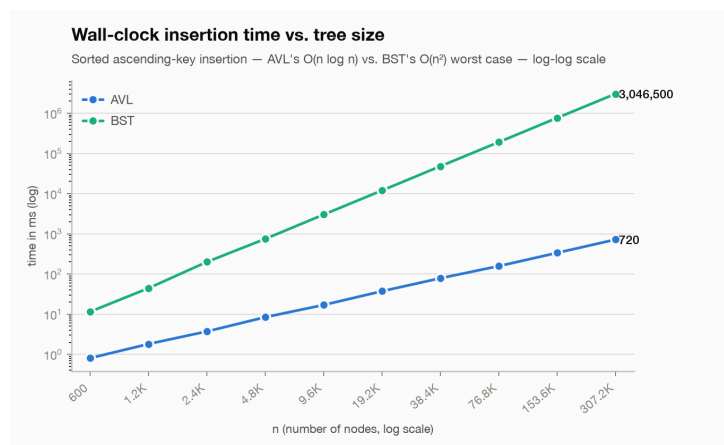
4. Question 4

4.1. Conclusion about Q 3

The thing that surprised us the most with the results of the experiments is how long it took for the sorted insertion in a regular BST tree. With all things considered, it has a $O(n^2)$ time complexity which should imply that it would take a long time, but reaching 50 minutes for inserting 300K nodes is not something that can be used in the modern industry.

The AVL Tree insertion was much faster. We saw that in the complexity analysis in class and on the experiments. Even in the worst case, of inserting keys in a sorted way, the insertion was, to an extent, pretty good. Reaching less then a second to insert 300K nodes.

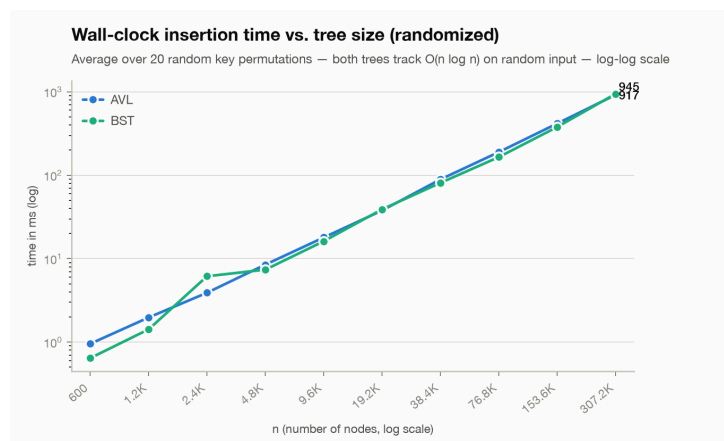
We added a graph that would emphasize the time different between inserting to a regular BST and an AVL Tree:



Although these results show a massive difference between inserting **sorted nodes** to an AVL Tree and a regular BST, the gap between inserting nodes in a randomized order.

After averaging 20 random permutations of $1, 2, \dots, n$ for various values of n , the results showed us there isn't the much of gap between those two BST implementations.

These are the results of the averages for both a regular BST and an AVL Tree in a graph:



4.2. Comparison to Q1, Q2

The results of the experiments did support our claims really well. We saw the $O(n^2)$ complexity for the worst case for inserting into a regular BST in action, while facing the real problem which is waiting for the program to finish, and when compared to an AVL Tree we saw a massive improvement. Which supports the fact the even in the worst case, inserting into an AVL Tree we get an $O(n \log(n))$ complexity

4.3. Classroom Claims

4.3.1.

The claim was not supported by our experimental results. In our experiments, we measured the total number of rotations performed for a given value of n , rather than examining the number of rotations performed during each individual insertion. Consequently, our methodology did not evaluate how many rotations occurred in a single insertion.

It is possible for one insertion to require, for example, ten rotations, while the following ten insertions require none. To verify the claim, one would need to record the number of rotations after each insertion and determine whether it exceeds one or two, depending on the type of rotation being considered. Moreover, no experiment can conclusively prove this claim, since it is asserted to hold for every possible value of n , whereas experiments can only be conducted for a finite set of values.

Furthermore, our implementation does not support performing more than one rotation per insertion, as this is unnecessary according to the AVL tree insertion algorithm presented in class.

4.3.2.

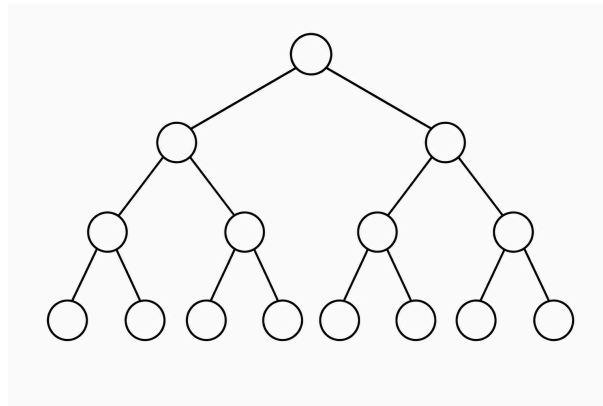
The claim is supported by our experimental results. We observe that the ratio between the total rebalancing work (including both rotations and height changes) and the number of insertions approaches a constant value as n tends to infinity (approximately 3 for sorted insertions and 2.48 for random insertions). Therefore, the amortized cost of rebalancing per insertion remains constant, consistent with the claim.

However, no experiment can conclusively prove this claim, since it is asserted to hold for every possible value of n , whereas experiments can only be conducted for a finite set of values.

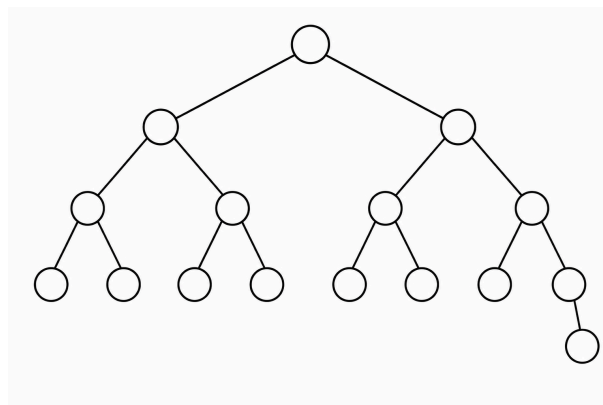
5. Question 5

5.1. Prove

We'll describe an edge case where a series of actions, which are insertions and deletions, that, through rebalancing, will need to update the height of each node from the new create/deleted up to the root, resulting in a $\Omega(\log(n))$ time complexity for each one of those actions (both insertion and deletion). The edge case will start with a perfect tree that holds $2^n - 1$ nodes which will look like this:



Now if we will add a new nodes (we'll add the max value for simplicity reasons) it will look like this:



For this insertion the height of the root (and each node from the new inserted node up to the root) is increased by 1, thus during the rebalancing we will need to “climb” all the way up to the root to update the heights and find out that there's no need for any rotations. This causes each insertion rebalance to take $\Omega(\log(n))$ time complexity

In a completely symmetric way, after each insertion, we will delete that exact node, which will cause the height to decrease back by 1 for each node from the parent of the deleted node up to the root node. Resulting in another $\Omega(\log(n))$ time complexity.

After the deletion we will start over with another insertion and deletion until we reach n actions. Each action takes $\Omega(\log(n))$ time complexity resulting in a total of $\Omega(n \log(n))$ for the entire sequence.

5.2. Experiment

5.2.1. Description

In our experiment, we built a perfect tree with height n and $2^n - 1$ nodes, starting from 1. Then for $2^n - 1$ iterations, we insert 2^n and then delete him. Each time counting the numbers of height changes in the insertion. We repeat this process for $n = 1, \dots, 15$ and computing the ratio between the total number of height changes to the total number of actions.

The total rebalancing work for a given n is greater than the total height changes in the insertion method. Therefore if the ratio between the total number of height changes to the total number of actions is not constant, so the ratio between the total rebalancing work to the number of action. Therefore we can not achieve $O(1)$ amortized.

This is our code:

```
for n in range(1, 16):
    keys = generate_perfect_bst_level_order(n)
    perfect_tree = AVLTree(is_avl=True)
    for key in keys:
        perfect_tree.insert(key, str(key))

    height_changes = 0
    actions = 0
    for i in range(2**n - 1):
        if i%2 == 0:
            height_changes += perfect_tree.insert(2**n, str(2**n))[3]
        else:
            perfect_tree.delete(perfect_tree.search(2**n)[0])
            actions += 1
    print(f"Perfect BST with {2**n-1} nodes: height changes after insert/delete = {height_changes}")
    print("Ratio of height changes to number of actions:", height_changes / actions)
```

5.2.2. Table

Iteration	Size	Height Changes	Ratio
1	1	1	1.00
2	3	4	1.33
3	7	12	1.71
4	15	32	2.13
5	31	80	2.58
6	63	192	3.04
7	127	448	3.52
8	255	1024	4.02
9	511	2304	4.51
10	1023	5120	5.00
11	2047	11264	5.50
12	4095	24576	6.00
13	8191	53248	6.50
14	16383	114688	7.00
15	32767	245760	7.50

5.2.3. Results

As we hypothesized the ratio is not constant and grows linearly as n grows exponential. In other words, the ratio grows at logarithmic rate.